

Worksheet No. - 03

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 3rd

Subject Name: Machine Learning Lab

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 01.09.2026

Subject Code: 25CAP-702

Aim/Overview of the practical:(A). Implement classification algorithms—Logistic Regression and Polynomial Regression—on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score.

Objective: To implement Logistic Regression and Polynomial Regression algorithms on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score, in order to understand and compare the effectiveness of regression-based machine learning techniques for prediction and classification tasks.

Procedure/Algorithm/Code

Logistic Regression

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score,
classification_report

# Load dataset
df = pd.read_csv("OneDrive/Desktop/Machine Learning/Sports Supplements.csv")

# Create binary target
target = 'evidence level - score. 0 = no evidence, 1,2 = slight, 3 = conflicting , 4 = promising, 5 = good, 6
= strong '
df['Target'] = (df[target] >= 4).astype(int)
# Select numerical features
features = [
    'popularity',
    'number of studies examined',
```

```
'number of citations',
'Cochrane systematic review',
'Other International review board or metastudy',
'No. of individual studies / reviews'
]
X = df[features]
y = df['Target']
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Create and train Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
cm = confusion_matrix(y_test, y_pred)
print("Logistic Regression Results")
print("Confusion Matrix:")
print(cm)
print("\nPrecision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score:", f1_score(y_test, y_pred, zero_division=0))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

Logistic Regression Results

Confusion Matrix:
[[17 0]
 [2 1]]

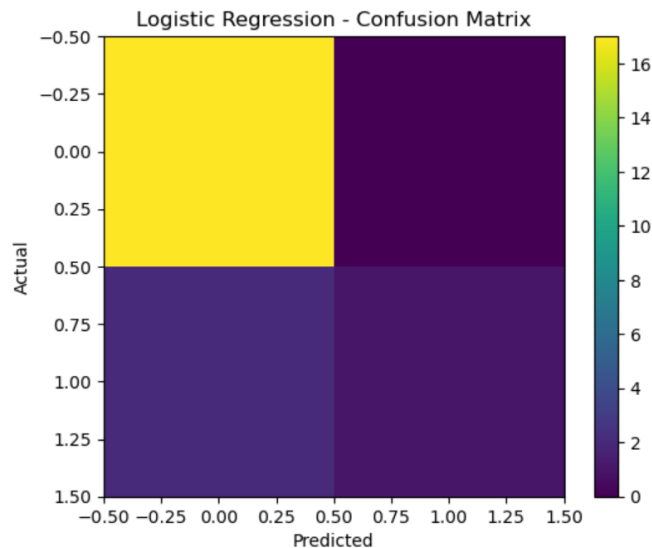
Precision: 1.0
Recall: 0.3333333333333333
F1-Score: 0.5

Classification Report:

	precision	recall	f1-score	support
0	0.89	1.00	0.94	17
1	1.00	0.33	0.50	3
accuracy			0.90	20
macro avg	0.95	0.67	0.72	20
weighted avg	0.91	0.90	0.88	20

```
# Display confusion matrix
plt.imshow(cm)
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```



Polynomial Regression

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score,
    classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load the dataset
df = pd.read_csv("OneDrive/Desktop/Machine Learning/Sports Supplements.csv")

# Step 3: Select suitable columns
```

```
score_column = df.columns[2]
data = df[['popularity', score_column]].copy()
data.columns = ['popularity', 'evidence_score']
```

Step 4: Convert columns to numeric

```
data['popularity'] = pd.to_numeric(
    data['popularity'],
    errors='coerce'
)
data['evidence_score'] = pd.to_numeric(
    data['evidence_score'],
    errors='coerce'
)
```

Step 5: Remove missing values

```
data = data.dropna()
print("\nCleaned Dataset:")
print(data.head())
print("\nNumber of records:", len(data))
```

```
Cleaned Dataset:
   popularity  evidence_score
0          242             1.0
1          242             0.0
2          947             5.0
3         5490             4.0
4         5490             3.0
```

```
Number of records: 96
```

Step 6: Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 76
Testing samples: 20
```

Step 7: Create Polynomial Regression model

Degree = 2 means quadratic polynomial

```
model = make_pipeline(  
    PolynomialFeatures(degree=2),  
    LinearRegression()  
)
```

Step 8: Train the model

```
model.fit(X_train, y_train)
```

Step 9: Predict evidence scores

```
y_pred = model.predict(X_test)  
print("\nActual Evidence Scores:")  
print(y_test.values)
```

```
print("\nPredicted Evidence Scores:")  
print(np.round(y_pred, 2))
```

```
Actual Evidence Scores:  
[2. 1. 0. 3. 3. 0. 0. 0. 1. 2. 3. 0. 1. 3. 1. 5. 1. 4. 1. 2.]
```

```
Predicted Evidence Scores:  
[1.09 1.54 1.05 2.55 1.31 1.09 1.09 2.68 0.92 1.12 1.9  1.8  0.9  2.16  
 1.17 1.51 1.16 2.41 1.05 0.92]
```

Step 10: Regression Evaluation

```
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
r2 = r2_score(y_test, y_pred)  
print("REGRESSION PERFORMANCE")
```

```
print("MAE :", round(mae, 4))  
print("MSE :", round(mse, 4))  
print("RMSE :", round(rmse, 4))  
print("R² :", round(r2, 4))
```

REGRESSION PERFORMANCE

MAE : 1.0413

MSE : 1.8339

RMSE : 1.3542

R² : 0.0955

Step 11: Convert regression output into classes

```
y_test_class = np.where(
    y_test >= 3,
    'Strong/Promising',
    'Limited/Conflicting'
)
y_pred_class = np.where(
    y_pred >= 3,
    'Strong/Promising',
    'Limited/Conflicting'
)
```

Step 12: Display classification results

```
print("CLASSIFICATION RESULTS")
print("\nActual Classes:")
print(y_test_class)
```

```
print("\nPredicted Classes:")
print(y_pred_class)
```

Step 13: Confusion Matrix

```
labels = [
    'Limited/Conflicting',
    'Strong/Promising'
]
cm = confusion_matrix(
    y_test_class,
    y_pred_class,
    labels=labels
)
print("\nConfusion Matrix:")
print(cm)
```

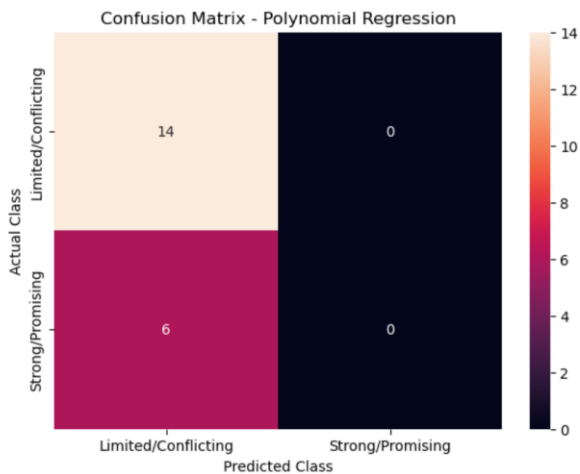
Step 14: Plot Confusion Matrix

```
plt.figure(figsize=(7, 5))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    xticklabels=labels,
    yticklabels=labels
)
```

```
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Polynomial Regression")

plt.show()
```



Step 15: Calculate Accuracy

```
accuracy = accuracy_score(
    y_test_class,
    y_pred_class
)
print("\nAccuracy:", round(accuracy, 4))
```

Step 16: Calculate Precision

```
precision = precision_score(
    y_test_class,
    y_pred_class,
    pos_label='Strong/Promising',
    zero_division=0
)
print("Precision:", round(precision, 4))
```

Step 17: Calculate Recall

```
recall = recall_score(
    y_test_class,
    y_pred_class,
    pos_label='Strong/Promising',
```

```
zero_division=0
)
print("Recall:", round(recall, 4))
```

Step 18: Calculate F1-Score

```
f1 = f1_score(
    y_test_class,
    y_pred_class,
    pos_label='Strong/Promising',
    zero_division=0
)
print("F1-Score:", round(f1, 4))
```

Step 19: Classification Report

```
print("CLASSIFICATION REPORT")
print(
    classification_report(
        y_test_class,
        y_pred_class,
        zero_division=0
    )
)
```

CLASSIFICATION REPORT				
	precision	recall	f1-score	support
Limited/Conflicting	0.70	1.00	0.82	14
Strong/Promising	0.00	0.00	0.00	6
accuracy			0.70	20
macro avg	0.35	0.50	0.41	20
weighted avg	0.49	0.70	0.58	20

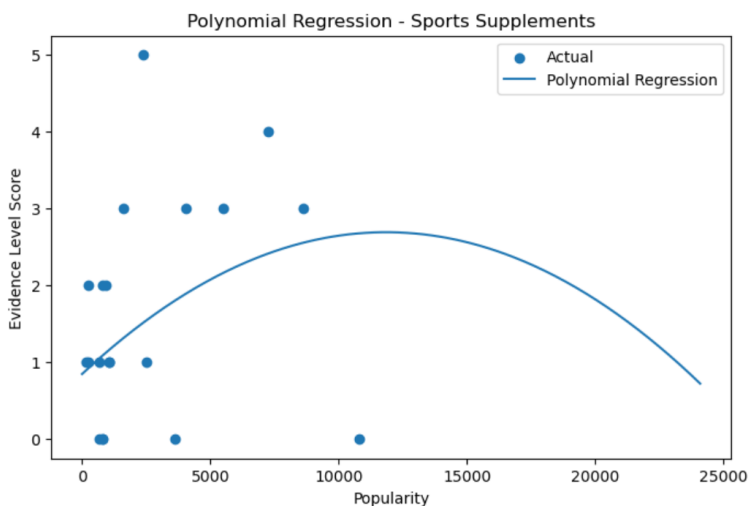
Step 20: Plot Polynomial Regression Curve

```
X_curve = np.linspace(
    X['popularity'].min(),
    X['popularity'].max(),
    300
).reshape(-1, 1)

y_curve = model.predict(
    pd.DataFrame(
        X_curve,
        columns=['popularity']
    )
)
```



```
plt.figure(figsize=(8, 5))
plt.scatter(
    X_test['popularity'],
    y_test,
    label='Actual'
)
plt.plot(
    X_curve,
    y_curve,
    label='Polynomial Regression'
)
plt.xlabel("Popularity")
plt.ylabel("Evidence Level Score")
plt.title("Polynomial Regression - Sports Supplements")
plt.legend()
plt.show()
```



Learning outcomes (What I have learnt):

1. Understood and implement Polynomial Regression for predicting continuous values.
2. Applied data preprocessing and prepare a suitable dataset for model training.
3. Divided the dataset into training and testing sets for model evaluation.
4. Transformed input features into polynomial features and train a regression model.